

Q23. Graph Partitioning Problem (NP-Hard / Constraint Optimization Problem)

Name: Moshitha Kolanjiyappan

Register No: 192421205

CO5-AT1 – Constraint-Based Problem Solving

Course Code: CSA0603 / DAA

1. Introduction

The Graph Partitioning Problem is an important constraint optimization problem in computer science. The objective is to divide the vertices of a graph into a specified number of subsets while minimizing the number or total weight of edges that cross between different subsets. Such crossing edges are called edge cuts. At the same time, the subsets should remain balanced according to a given constraint, such as having approximately the same number of vertices or approximately the same total vertex weight. Graph partitioning is widely used in parallel computing, distributed systems, VLSI design, image processing, network design, clustering, and database systems.

Let a graph be represented as $G = (V, E)$, where V is the set of vertices and E is the set of edges. A partition divides V into k non-empty disjoint subsets $P_1, P_2, \dots, P_k$ such that every vertex belongs to exactly one subset. The optimization goal is to obtain a balanced partition with a small cut size. A good solution therefore has two characteristics: it satisfies all required constraints and keeps strongly connected vertices in the same subset whenever possible.

2. Problem Definition

Given an undirected graph $G = (V, E)$, partition its vertices into k subsets. For an edge (u, v) , the edge is cut if u and v belong to different subsets. The basic objective can be written as minimizing the cut cost:

$$\text{Cut}(P) = \sum w(u,v), \text{ for every edge } (u,v) \text{ such that } \text{part}(u) \neq \text{part}(v).$$

If all edges have equal weight, $w(u,v) = 1$, and the objective is simply to minimize the number of crossing edges. In a weighted graph, an edge with a larger weight represents a stronger relationship, communication cost, or dependency, so cutting it should be penalized more heavily.

The partition must also satisfy a balance constraint. For example, if there are n vertices and k partitions, an exact balance condition may require each partition to contain n/k vertices. A relaxed balance condition can allow a tolerance δ . Then the size of each partition should satisfy a

lower and upper bound. For weighted vertices, the same idea can be applied to the sum of vertex weights rather than just the number of vertices.

3. Constraints

The main constraints of the problem are as follows.

1. Assignment constraint: Every vertex must be assigned to exactly one partition. No vertex can be omitted or placed in multiple subsets.
2. Non-empty constraint: Each partition must contain at least one vertex unless an empty partition is explicitly allowed.
3. Balance constraint: The sizes of the partitions must remain within the specified lower and upper bounds. For example, with n vertices and k partitions, an exact balanced solution has approximately n/k vertices per partition.
4. Cut constraint/objective: Edges connecting vertices placed in different partitions contribute to the cut cost. The objective is to minimize this value rather than treating it as an independent feasibility requirement.
5. Optional capacity constraint: If partitions represent machines or processors, each partition may have a maximum capacity. The total weight assigned to a partition must not exceed that capacity.

These constraints make the problem more realistic but also reduce the number of feasible solutions. The balance condition is especially important because a solution with one very large group and several tiny groups could have a very small cut, but it would not be useful for parallel or distributed execution.

4. Effect of Constraints on Partition Quality

Constraints directly affect the quality of the final partition. If there are no balance constraints, the algorithm may place most or all highly connected vertices in one subset. This can reduce the cut dramatically, but the workload becomes uneven. One processor may receive most of the work while other processors remain underutilized.

A strict balance constraint improves workload distribution but can force some strongly connected vertices into different partitions. This increases the cut cost. Therefore, graph partitioning involves a trade-off between balance and connectivity. If the balance tolerance is too small, the algorithm has very little freedom and may produce a higher cut. If the tolerance is increased, more candidate partitions become feasible and the algorithm may find a lower-cut solution.

For weighted graphs, the effect is even clearer. Cutting a high-weight edge can be much worse than cutting several low-weight edges. Thus, a weighted objective allows the algorithm to preserve important relationships while accepting less expensive cuts elsewhere. In practice, the best partition is not necessarily the one with the absolute minimum cut; it is the best feasible solution under the required balance and capacity constraints.

5. Backtracking Solution

A backtracking algorithm can systematically explore possible assignments of vertices to partitions. At each step, the next unassigned vertex is placed into one of the k partitions. After an assignment, the algorithm checks whether the partial solution can still satisfy the balance constraints. If it violates a capacity or upper-bound condition, that branch is immediately rejected.

The general procedure is:

Step 1: Start with all vertices unassigned.

Step 2: Select the next vertex. A useful heuristic is to choose a high-degree vertex first because its assignment can have a larger effect on the cut.

Step 3: Try assigning the vertex to each feasible partition.

Step 4: Update the current partition sizes and the partial cut cost.

Step 5: Check the balance and capacity constraints. If a constraint is violated, undo the assignment and try another partition.

Step 6: Calculate a lower bound on the best possible final cut from the current partial assignment.

Step 7: If this lower bound is already greater than or equal to the best complete solution found so far, prune the branch.

Step 8: Continue until all vertices are assigned.

Step 9: Store the feasible assignment with the minimum cut cost.

This approach is exact when every relevant assignment is explored except branches proven unable to improve the current best solution. However, its worst-case running time remains exponential.

6. Pruning Strategy

Pruning is essential because blindly exploring all assignments is extremely expensive. Several pruning rules can reduce the search space.

The first rule is feasibility pruning. If a partition already exceeds its maximum permitted size, the current branch cannot become valid, so it is discarded immediately.

The second rule is capacity pruning. If the remaining unassigned vertices cannot possibly fill the minimum required sizes of the remaining partitions, the branch is infeasible and can be stopped.

The third rule is lower-bound pruning. Suppose the current partial assignment has a cut cost C . Even if all remaining vertices could be assigned in the most favorable possible way, a lower bound LB can be estimated for additional unavoidable cuts. If $C + LB$ is not smaller than the best solution already obtained, there is no reason to explore that branch.

The fourth rule uses symmetry. Partition labels are interchangeable. For example, assigning the first vertex to partition 1 is equivalent to assigning it to partition 2 after simply renaming the partitions. To avoid exploring equivalent states, the first vertex can be fixed in the first partition, and new partition labels can be introduced in a fixed order.

A heuristic ordering also improves pruning. High-degree or high-weight vertices are assigned first. This tends to produce a good complete solution early, giving a strong upper bound and allowing more branches to be pruned.

7. Heuristic Alternative

For larger graphs, a heuristic method is usually more practical than exact backtracking. A simple greedy heuristic begins by assigning vertices one at a time to the partition that causes the smallest increase in cut cost while respecting balance constraints.

For each unassigned vertex v and candidate partition P_j , the algorithm calculates the additional cut cost. If a neighbor of v is already in P_j , that edge does not become a cut. If the neighbor is in another partition, the edge contributes to the cut. The vertex is placed in the partition with the best feasible score.

A local improvement stage can then be applied. The algorithm examines whether moving a vertex from one partition to another decreases the cut while maintaining balance. If such a move exists, it is performed. The process continues until no improving move is available.

This heuristic does not guarantee the global optimum, but it is much faster and can handle graphs that are too large for exhaustive backtracking. A useful practical strategy is to generate an initial

solution using the heuristic and then use its cut cost as the initial upper bound for a backtracking or branch-and-bound search.

8. Example

Consider a graph with 8 vertices and two partitions, A and B. Suppose the required balance is exactly four vertices in each partition. Assume the graph contains several edges among vertices 1, 2, 3, and 4, and several edges among vertices 5, 6, 7, and 8, with only a few edges connecting these two groups.

A natural partition is $A = \{1,2,3,4\}$ and $B = \{5,6,7,8\}$. It satisfies the balance constraint because both groups contain four vertices. If only two edges connect the two groups, the cut size is 2.

Now imagine removing the balance constraint. An algorithm might place seven vertices in A and one vertex in B, potentially reducing the cut to 1 or even 0 in a specially structured graph. Although this may look better according to the cut objective, it is not a valid balanced solution. This example shows why constraints are necessary: optimization without feasibility requirements can produce solutions that are mathematically attractive but practically useless.

In a weighted version, suppose the two crossing edges have weights 2 and 10. The cut cost is 12. The algorithm should prefer an alternative balanced partition if it can replace the weight-10 cut with several much smaller cuts. Therefore, the objective must consider edge weights whenever relationships have different importance.

9. Constraint Satisfaction and Correctness

The solution satisfies the constraints because every assignment is checked before it is accepted. The assignment constraint is guaranteed by maintaining exactly one partition value for each vertex. Non-empty partitions are checked when a complete solution is evaluated. Balance is enforced using minimum and maximum allowed partition sizes. Capacity constraints, when present, are checked after every assignment.

The cut objective is computed only for edges whose endpoints are assigned to different partitions. The final solution is selected only from feasible complete assignments. Therefore, the reported result is not simply the partition with the smallest cut; it is the partition with the smallest cut among those satisfying the constraints.

Pruning does not affect correctness when it is based on valid feasibility tests or valid lower bounds. A branch is removed only when it is impossible to produce a feasible solution or impossible to improve the current best solution. Consequently, an exact branch-and-bound implementation still returns an optimal feasible partition.

10. NP-Hard Complexity

Graph partitioning is computationally difficult. Important forms of balanced graph partitioning, including minimum bisection, are NP-hard. The difficulty comes from the enormous number of possible assignments. For n vertices and k labeled partitions, there can be up to k^n assignments before balance constraints and symmetry are considered. Even for two partitions, the number of possible assignments is exponential.

Backtracking can reduce practical work through pruning, but its worst-case time complexity remains exponential. A simplified upper bound is $O(k^n)$ in the absence of effective pruning. The exact complexity depends on the graph, number of partitions, balance requirements, and pruning rules.

Because NP-hard problems are unlikely to have polynomial-time exact algorithms in general, real applications often use approximation algorithms, greedy methods, local search, multilevel partitioning, spectral techniques, or specialized solvers. The choice depends on whether optimality, execution time, graph size, or solution quality is the most important requirement.

11. Conclusion

The Graph Partitioning Problem demonstrates the role of constraint-based problem solving in an NP-hard optimization setting. The objective is to minimize edge cuts while maintaining balanced and feasible partitions. Balance, capacity, assignment, and non-empty constraints prevent the algorithm from selecting impractical solutions. At the same time, strict constraints can increase the minimum achievable cut, creating a fundamental trade-off between partition balance and connectivity.

A backtracking or branch-and-bound method can find an optimal solution for small and medium instances by exploring assignments and applying feasibility, lower-bound, and symmetry pruning. For larger graphs, a greedy heuristic followed by local improvement provides a practical alternative. Combining a heuristic initial solution with backtracking can be particularly effective because the heuristic supplies a good upper bound for pruning.

Thus, graph partitioning is an excellent example of how constraints define the feasible search space, optimization determines the preferred solution, and pruning makes an otherwise exponential search more manageable. Although the underlying problem is NP-hard, carefully designed constraints, heuristics, and pruning techniques can produce high-quality solutions for practical applications.